DP-PFN: Differentially Private Prior-Data Fitted Networks

Nov 5, 2026 · @Karen Eze

Summary

DP-PFN is feasible and has a sound theoretical basis, because PFN weights cost no privacy budget. The central caveat: a trained PFN cannot be certified from its training loss. Guarantees must therefore come from black-box wrappers or from noise-aware use of already-private data.

- **Q1, theory (Sections 1–6).**
  - **Free weights.** Only the context costs budget (Prop. 0).
  - **Exact route.** One draw from the exact PPD is 2B-DP. Floored priors make B finite and can be simulated. A clean-target PFN releases many labels for the cost of one draw.
  - **Amortization gap.** Average-case training cannot certify worst-case DP (Prop. B2).
  - **Black-box route.** Three mechanisms: Vote, LogPool and Student. Theorem C2 links the DP sample size to teacher accuracy on small shards.
  - **Noise-aware route.** Put the mechanism inside the prior. The PFN then analyses DP data Bayes-optimally, with privacy by post-processing (Thm. D1).
  - **Graphs.** Label-DP via graph-shard voting; edge-DP via noisy sum aggregation inside the PFN. Under edge randomized response a cSBM stays a cSBM. Node-DP stays open.
- **Q2, backbones (Section 7).**
  - **TabPFN:** usable now inside the wrappers, with shard-local preprocessing and synthetic-only checkpoints.
  - **NodePFN:** usable now for label-DP. Edge-DP needs sum aggregation, shared ensemble aggregates and noise-aware retraining.
- **Q3, advantages (Section 8).**
  - **Stronger:** no privacy spent on training or tuning; public knowledge without public data; smaller DP sample size on small data; calibrated inference from noisy releases; no per-graph training.
  - **Weaker:** certification, per-query cost, context size and node-DP.

- **Q4, plan (Sections 9–11).** Three papers over about 14 months, tabular first. Nine theory items and eight experiments, with three go/no-go gates.

1. Setup: what is private and what is free

In DP-PFN the network weights depend only on synthetic prior data. They therefore cost no privacy budget, and every privacy question concerns the context D and what is released from it.

Records and prior. A record is z = (x, y) with x in a feature space X and y in K classes. The context is D = (z_1, ..., z_n), and a query is x*. The prior draws a task parameter φ ~ π, then records i.i.d. from p(z | φ).

Target and objective. The posterior predictive distribution (PPD) and the PFN training loss are:

```
P(y \mid x^*, D) = \int p(y \mid x^*, \varphi)\, \pi(\varphi \mid D)\,
d\varphi, \qquad \pi(\varphi \mid D) \propto \pi(\varphi) \prod_{i=1}^{n}
p(z_i \mid \varphi)
```

```
\mathcal{L}(\theta) = \mathbb{E}_{\varphi, D, x^*, y^*}\big[-\log
q_\theta(y^* \mid x^*, D)\big] = \mathbb{E}_{D, x^*}\,
\mathrm{KL}\big(P(\cdot \mid x^*, D)\,\|\, q_\theta(\cdot \mid x^*, D)\big) +
\text{const}
```

The second identity is the PFN theorem of Müller et al. (2022). It matters below: the PFN is only trained to be close to the PPD **on average over prior-drawn datasets**.

Neighbouring relations. DP-PFN must say which change to the data it protects against.

Setting	Neighbours D ≃ D′ differ in	Protects
Tabular row-DP (replace-one)	one row (x_i, y_i)	a whole record
Tabular label-DP	one label y_i, features fixed	labels only; features treated as public
Graph edge-DP	one edge	a relationship
Graph node-DP	one node with its features, label and all incident edges	everything about one person

Setting	Neighbours $D \simeq D'$ differ in	Protects
Graph label-DP	one node label	labels only

Release modes. We study three ways outputs leave the system.

- **R1, prediction API.** k possibly adaptive queries are answered from D. Privacy composes over k.
- **R2, student-context release.** One privately labelled context is published once. Anyone can then run the public PFN on it, as post-processing, with no further cost.
- **R3, noise-aware release.** A DP summary $S = M(D)$ is published (noisy statistics, randomized-response labels, DP synthetic rows, noisy graph aggregates). A PFN trained to read S does the inference.

Proposition 0 (free weights). Suppose θ is statistically independent of D. Then for any mechanism A, the map $D \mapsto A(q_\theta, D)$ has exactly the privacy of A run with q_θ as a fixed public function. All accounting therefore concerns the context alone.

Proposition 0 fails in two practical cases, and DP-PFN must rule both out:

1. **Real-data checkpoints.** Weights fine-tuned on real tables are no longer free. DP-PFN should use synthetic-only checkpoints.
2. **Data-dependent preprocessing.** Standardization, quantile transforms or PCA fitted on the private context are themselves releases. They must use public feature bounds or be privatized and counted in the budget.

The prior designer must also not tune the prior on the private dataset, since that would make θ depend on D.

2. Theory A: one draw from the exact PPD is already DP

If every record's likelihood has bounded oscillation B under every task, one draw from the exact PPD is 2B-DP. B is finite only if the prior is designed that way. That design can be simulated, so a PFN can be trained on it.

Definition A1 (oscillation). For records and for labels:

```
B = \sup_{\varphi}\Big[\sup_{z}\log p(z\mid\varphi) - \inf_{z}\log
p(z\mid\varphi)\Big], \qquad B_y = \sup_{\varphi, x}\Big[\max_{y}\log p(y\mid
x,\varphi) - \min_{y}\log p(y\mid x,\varphi)\Big]
```

Theorem A2 (posterior and PPD sampling). Let D ≃ D′ be replace-one neighbours and B < ∞. Then for all φ, x* and y:

```
\pi(\varphi\mid D) \le e^{2B}\,\pi(\varphi\mid D'), \qquad P(y\mid x^*, D)
\le e^{2B}\, P(y\mid x^*, D')
```

So releasing φ ~ π(· | D), or y ~ P(· | x*, D), or any function of either, is 2B-DP. For label neighbours the same holds with B_y.

Proof. Let D′ replace z_j with z_j′, and let L(φ) be the product of the other n − 1 likelihoods.

```
\frac{\pi(\varphi\mid D)}{\pi(\varphi\mid D')} = \frac{p(z_j\mid\varphi)}
{p(z_j'\mid\varphi)}\cdot\frac{Z(D')}{Z(D)}, \quad Z(D') = \int \pi L\,
p(z_j'\mid\varphi)\, d\varphi \le e^{B}\int \pi L\, p(z_j\mid\varphi)\,
d\varphi = e^{B} Z(D)
```

Both factors are at most e^B. The PPD mixes the same kernel p(y | x*, φ) over the posterior, so the bound carries over. If x* is itself informative about φ, condition the posterior on x* as well; x* is common to both sides, so the bound is unchanged. ∎

This is the "privacy for free" result of Dimitrakakis et al. (2014) and Wang, Fienberg & Smola (2015). They state 4B because they bound |log p| ⩽ B, which is an oscillation of 2B.

Why standard PFN priors give B = ∞.

- **Features.** Priors that generate x and y jointly from a structural causal model (as TabPFN's does) use continuous feature densities. Their log-density is unbounded, so B = ∞ for row-DP.
- **Labels.** If any class probability can approach 0, then B_y = ∞.

Definition A3 (floored prior). Mix each likelihood with a strictly positive reference. Here r is uniform on a bounded feature domain, and p(z | φ)/r(z) ⩽ M.

```
p_\alpha(z\mid\varphi) = (1-\alpha)\,p(z\mid\varphi) + \alpha\, r(z), \qquad
p_\eta(y\mid x,\varphi) = (1-\eta)\,p(y\mid x,\varphi) + \eta/K
```

Lemma A4. B(p_α) ⩽ log(1 + (1 − α)M/α) and B_y(p_η) ⩽ log(1 + K(1 − η)/η).

Floors can be simulated; tempering cannot. A floored likelihood is a mixture, so data can be drawn from it: with probability η, resample the label uniformly. A PFN can therefore be trained on the floored prior, with η as an input token, which makes ε a dial set at inference time. A tempered posterior π(φ) Π p(z_i | φ)^τ corresponds to no data-generating process, so a PFN cannot be trained on it directly.

η (label resample prob.)	Flip rate (K = 2)	B_y	ε = 2B_y	ε after subsampling, m/n = 0.1
0.2	10%	2.20	4.39	2.20
0.5	25%	1.10	2.20	0.59
0.8	40%	0.41	0.81	0.12

Theorem A5 (a clean-target PFN draws one posterior sample). Train with η-noisy context labels but clean query targets $h_\varphi(x^*) = \operatorname{argmax}_y p(y \mid x^*, \varphi)$. The loss minimizer is $Q(y \mid x^*, D) = \pi(h_\varphi(x^*) = y \mid D)$. Sampling from Q is post-processing of one posterior draw, hence $2B_y$-label-DP. Unlike sampling the noisy PPD, Q contains no η label noise. Where the posterior is consistent, Q concentrates on the Bayes decision as n grows, while ε stays fixed. T1 (Section 9) asks how fast, when η misstates the real label noise.

Theorem A6 (many labels for the price of one). Let the PFN also accept context tokens flagged "clean pseudo-label". At public query points $x_1, \ldots, x_m$, sample $\hat{y}_1 \sim Q(\cdot \mid x_1, D)$, then $\hat{y}_2 \sim Q(\cdot \mid x_2, D, (x_1, \hat{y}_1))$, and so on. If every conditional is exact, the chain rule gives:

$$(\hat y_1,\dots,\hat y_m) \overset{d}{=} \big(h_\varphi(x_1),\dots,h_\varphi(x_m)\big), \quad \varphi \sim \pi(\cdot\mid D)$$

The whole labelled student context is post-processing of one posterior draw. It costs $2B_y$ in total, whatever m is. This is an exact-case DP reading of predictive resampling, as in martingale posteriors (Fong, Holmes & Walker, 2023). In release mode R2, it makes cost independent of the student-context size, whereas noisy-vote labelling (PATE) pays per label.

Theorem A7 (amplification by subsampling). Run any of the above on m records drawn without replacement from n. An ε-DP base mechanism becomes $\log(1 + (m/n)(e^\varepsilon - 1))$-DP (Balle, Barthe & Gaboardi, 2018). PFNs take subsampled contexts natively, so this costs nothing extra. The last column of the table above uses it.

Exactly solvable test beds. Beta–Bernoulli, Dirichlet–categorical and Gaussian-process priors have closed-form posteriors. Geumlek, Song & Chaudhuri (2017) give tight Rényi-DP bounds for exponential families. These let us compare a trained PFN with the exact mechanism (Section 3).

All the ε values here hold for the **exact** posterior. Section 3 shows why a trained PFN does not inherit them automatically.

3. Theory B: the amortization gap

A PFN is trained to match the PPD on average, while DP is a worst-case condition. So no bound on training loss can certify that a trained PFN is DP. A certificate must come from the network's own row sensitivity, or from a mechanism whose guarantee does not depend on the network at all.

Per-dataset approximation error.

```
\Lambda(D) = \sup_{x^*, y}\big|\log q_\theta(y\mid x^*, D) - \log P(y\mid
x^*, D)\big|, \qquad \Delta(D) = \sup_{x^*}\mathrm{TV}\big(q_\theta(\cdot\mid
x^*, D),\, P(\cdot\mid x^*, D)\big)
```

Lemma B1 (transfer). Suppose sampling from the exact PPD is ε-DP.

- If sup_D Λ(D) ⩽ λ, sampling from q_θ is (ε + 2λ)-DP.
- If sup_D Δ(D) ⩽ γ, sampling from q_θ is (ε, (1 + e^ε)γ)-DP.

Proof of the second. For any event S: q(S | D) ⩽ P(S | D) + γ ⩽ e^ε P(S | D′) + γ ⩽ e^ε q(S | D′) + (1 + e^ε)γ. ∎

Proposition B2 (training loss cannot certify DP). The loss L(θ) sees q_θ only on datasets of positive prior probability. Changing q_θ on any prior-null set of datasets leaves L(θ) unchanged. With continuous features every single dataset is prior-null, including any pair D, D′ an adversary chooses. So for every attainable loss value, some predictor attains it and reveals a row deterministically on a chosen D, violating (ε, δ)-DP for every finite ε and δ < 1.

Even on a set A of positive mass ν, replacing q by (1 − κ)·(point mass) + κ·P on A raises the loss by at most ν log(1/κ). The sup in Lemma B1 is therefore something the training objective never controls.

Proposition B3 (certify the network, not its accuracy). Define the network's own row sensitivity:

```
s(\theta, n) = \sup_{D\simeq D',\, x^*,\, y}\big|\log q_\theta(y\mid x^*, D)
- \log q_\theta(y\mid x^*, D')\big|
```

Sampling from q_θ is then s-DP, with no reference to the PPD.

- **Output floor (trivial regime).** Mixing the output with uniform η/K gives s ⩽ log(1 + K(1 − η)/η) for any change to the whole context. This is randomized response on the prediction. It is private, but ε never shrinks with n.
- **Useful regime.** We want s = O(C/n), so each row's influence decays as the context grows. Consider one attention readout whose logits lie in [−R, R], for example by LayerNorm plus spectral-norm limits. Each attention weight is then at most e^{2R}/n, and replacing one row moves the readout by O(V e^{2R}/n), where V bounds the value norms. Layers in which context rows attend to each other compound the constant roughly geometrically in depth. Dot-product attention is not globally Lipschitz (Kim, Papamakarios & Mnih, 2021), so bounded inputs or L2 attention are needed.

Certified constants for a full TabPFN-scale network will be loose. B3 is realistic for a small "certifiable DP-PFN" used in theory, and as a basis for data-dependent methods such as propose-test-release.

Corollary B4 (design rule). Place the PFN inside a mechanism whose guarantee holds for any black box (Section 4), or let it read only a privatized summary (Section 5). Approximation error then costs utility, never privacy. Exact-PPD mechanisms (Section 2) serve as closed-form test beds and as the target that audits measure against.

Observation B5 (leakage partly reflects Bayes-optimality). Under a low-noise prior B_y is large, so the exact PPD on small n already depends strongly on single rows. A PFN that approximates it well inherits that dependence. Membership attacks on tabular foundation models therefore partly measure faithfulness to a non-private target. Examples are AMIA's attention-concentration signal (Carvalho & Cordy, 2026) and the attacks in TabPATE.

Prediction: better PFNs leak more unless the target itself is private. Experiment E1 (Section 10) tests this on closed-form priors, where the exact privacy loss is known.

4. Theory C: black-box DP-PFN mechanisms

The robust route treats the PFN as a free, near-Bayes-optimal teacher inside a mechanism whose guarantee holds for any black box. PFNs strengthen these mechanisms where they are weakest: small shards.

Mechanism C1 (DP-PFN-Vote, private prediction).

1. Split D at random into T disjoint shards of size m = n/T. The split must not depend on the data.

2. Teacher t predicts $\hat{y}_t = \text{argmax}_y\, q_\theta(y \mid x^*, D_t)$. This is one forward pass, with no training.
3. Release the noisy argmax of the vote counts $c_y(x^*)$.

Replacing one row changes one shard, so the vote histogram moves by at most 1 in two coordinates (ℓ_2 sensitivity $\sqrt{2}$). Report-noisy-max with Laplace($2/\varepsilon$) noise is ε-DP per answer. Gaussian noise of scale σ gives Rényi-DP α/σ^2 per answer, as in PATE's GNMax. Answers compose, and PATE's data-dependent analysis charges far less when teachers agree.

Theorem C2 (shard-level accuracy sets the DP sample size). Take binary labels. Disjoint shards are independent draws, so teachers are independent given the data distribution. Let $e(m)$ be one teacher's error at shard size m, with margin $\gamma_m = 1 - 2e(m) > 0$. Then C1 with Laplace($2/\varepsilon$) noise answers correctly with probability at least $1 - \beta$ whenever:

$$T \;\ge\; 8\log(2/\beta)\cdot\max\Big\{\frac{1}{\gamma_m^2},\; \frac{1}{\varepsilon\,\gamma_m}\Big\}, \qquad n^* = \min_m\; m\cdot T(m)$$

Proof. By Hoeffding, the vote margin falls below $T\gamma_m/2$ with probability at most $\exp(-T\gamma_m^2/8)$. Two Laplace($2/\varepsilon$) draws overturn a margin of $T\gamma_m/2$ with probability at most $\exp(-\varepsilon T\gamma_m/8)$. Set each term to $\beta/2$. ∎

Privacy enters only through T. A teacher that is more accurate at small m has a larger γ_m, which lowers n^*. PFNs are built for this small-data regime. PATE-style trained teachers degrade at small m, and each one must also be trained.

Mechanism C3 (DP-PFN-LogPool, a Bayes-consistent soft aggregator). Under the prior, shard posteriors combine exactly as $\pi(\varphi \mid D) \propto \pi(\varphi)^{1-T} \prod_t \pi(\varphi \mid D_t)$. The Bayesian committee machine (Tresp, 2000) applies the same rule to predictives. Floor each teacher's output at η/K, then sample from an exponential mechanism over the pooled score:

$$S_y(x^*) = \sum_{t=1}^{T} \log \tilde q_\theta(y\mid x^*, D_t) - (T-1)\log q_\theta(y\mid x^*, \varnothing), \qquad \Pr[\hat y = y] \propto \exp\!\Big(\frac{\varepsilon\, S_y}{2\log(K/\eta)}\Big)$$

The floored log-probabilities lie in an interval of width at most $\log(K/\eta)$. The prior-correction term does not depend on the data. So one shard change moves each S_y by at most $\log(K/\eta)$, and the sampler is ε-DP. As ε grows, it approaches the committee-machine argmax.

Unlike hard votes, LogPool keeps each teacher's confidence. That helps exactly when teachers are calibrated, which PFNs tend to be. When LogPool beats Vote is open question T4 (Section 9).

Mechanism C4 (DP-PFN-Student, release mode R2). Label N public or synthetic queries with C1 or C3, and publish them as a student context. Every later prediction is post-processing. TabPATE follows this design, with synthetic queries drawn from feature ranges. DP-PFN adds two options:

- Draw queries from the PFN's own prior, so query generation never touches the data.
- In the exact regime, replace per-label noisy votes with Theorem A6's single-draw cost.

Which mechanism when.

Mechanism	Holds for any network?	What costs privacy	Best when
C1 Vote	yes	each answered query (data-dependent analysis available)	few queries, teachers agree
C3 LogPool	yes, with output floor	each answered query	calibrated teachers, small K
C4 Student	yes	each labelled student query, once	many future queries
A5/A6 one-draw	exact PPD only	one posterior draw for all labels	closed-form priors; audited use
Section 5 noise-aware	yes (post-processing)	the release mechanism only	data already released under DP

5. Theory D: noise-aware DP-PFN

Suppose the data leave the owner through a public DP mechanism M. Putting M inside the prior turns the PFN into the Bayes-optimal analyst of the privatized data. Its privacy is exactly that of M, whatever the network's approximation error. This is the cleanest combination of the two theories.

Construction. Draw $\varphi \sim \pi$, $D \sim p(\cdot \mid \varphi)$, $S \sim M(D)$ and a query pair $(x^*, y^*) \sim p(\cdot \mid \varphi)$. Train $q_\theta(y^* \mid x^*, S)$, with S tokenized as the context. The mechanism's type and parameters (ε, σ, flip rate) are input tokens, so one network covers a family of mechanisms.

Theorem D1 (target, privacy, optimality).

1. The loss minimizer is the private-data PPD. Its posterior is intractable by classical means (Williams & McSherry, 2010; Bernstein & Sheldon, 2018; Ju et al., 2022).

$$P(y^*\mid x^*, S) = \int p(y^*\mid x^*, \varphi)\,\pi(\varphi\mid S)\, d\varphi, \qquad \pi(\varphi\mid S) \propto \pi(\varphi)\int M(S\mid D)\, p(D\mid\varphi)\, dD$$

2. Every $q_\theta(\cdot \mid x^*, S)$ inherits M′s (ε, δ) guarantee by post-processing, whatever its approximation error. Prediction queries are then free and unlimited.
3. Among all predictors that see only S, the target has the least expected log-loss under the prior. Its excess over the non-private Bayes predictor is exactly a conditional mutual information:

$$H(y^*\mid x^*, S) - H(y^*\mid x^*, D) = I(y^*; D\mid x^*, S) \;\ge\; 0$$

Proof of 3. M′s randomness is independent of everything else, so $S \perp y^*$ given (D, x^*). Then $I(y^*; D, S \mid x^*) = I(y^*; D \mid x^*)$, and the chain rule gives the identity. ∎

Consequence. For a fixed prior, $\varepsilon \mapsto I(y^*; D \mid x^*, S_\varepsilon)$ is the exact price of privacy for "release with M, then do anything". The noise-aware PFN attains this price, so a trained PFN can rank release mechanisms at equal ε by their downstream Bayes risk. That is open question T5.

Proposition D2 (naive plug-in is miscalibrated). Feeding DP outputs to a standard PFN targets $P(y^* \mid x^*, D = S)$, which treats noise as signal. The direction of the error depends on M:

- **Label randomized response** flattens the naive predictive toward uniform. With binary flip rate ρ it estimates $(1 - 2\rho)p + \rho$, so it is under-confident.
- **DP synthetic rows** generated in bulk make it over-confident, since every synthetic row is counted as real evidence.

The noise-aware target corrects both.

Mechanisms that can be simulated inside the prior.

Mechanism M	What the PFN reads	How the prior simulates it
Label randomized response (label-LDP)	context with noisy labels	flip with the RR probabilities at the given ε
Feature LDP (bounded features, Laplace or multi-bit)	context with noisy features	apply the same noise to simulated rows
Noisy marginals (Laplace or Gaussian)	a set of statistic tokens	compute marginals on simulated D, add noise
DP synthetic data (AIM, MST, PrivBayes)	synthetic rows	run the generator in simulation; costly, so use a calibrated surrogate or a small pool of runs
Graph: noisy aggregates, edge RR	noisy features and graph	see Section 6

Relation to prior work. Xiong, Ju & Zhang (TMLR 2025) fit a neural posterior estimator per model and per release. Noise-aware DP-PFN amortizes across datasets and tasks: one network serves every table released under a mechanism family.

Central versus local, a first comparison (open question T2). Take $K = 2$ and $\varepsilon \approx 2.2$. The central one-draw route of Section 2 must assume 25% label flips ($\eta = 0.5$). Local randomized response at the same ε actually flips about 10% ($1/(e^{2.2} + 1)$), with a noise-aware PFN downstream. But the OPS noise is assumed rather than injected, and its randomness shrinks with n. The comparison must therefore be made on accuracy, not flip rates.

6. Theory E: DP-PFN on graphs

On a graph, the PFN's context is a set of labelled nodes plus the whole graph. Privacy therefore splits by what changes: a label, an edge or a whole node. Label-DP and edge-DP have clean DP-PFN mechanisms. Node-DP remains hard.

What NodePFN computes. NodePFN (ICLR 2026) fuses three terms in every layer: $H^{\ell+1} = \text{LayerNorm}(H^{\ell} + \text{Attn} + \text{GCN}(H^{\ell}, \tilde{A}))$. Here $\tilde{A} = D^{-1/2} A D^{-1/2}$ is the symmetric-normalized adjacency. Context-node labels enter as an added label embedding, and test nodes cross-attend only to context nodes.

Two features of the pipeline also touch the graph: an optional feature-smoothing step that sums over edges, and a 32-member ensemble over feature order and scaling.

G1. Label-DP: graph-shard voting. Split the labelled nodes into T disjoint groups. Teacher t runs the PFN on the full graph and all features, which are public under label-DP, but uses only group t's labels as context. One label change moves one teacher, so Mechanism C1 and Theorem C2 apply unchanged.

On graphs, sharding splits only the labels: every teacher keeps all edges and features. Structural signal is not lost the way it is when tabular rows are sharded. For release mode R2, privately pseudo-label every unlabelled node once (a graph PATE), then run NodePFN freely on the result.

G2. Edge-DP: noisy aggregation inside the PFN. Replace each GCN step with a sum over unit-norm states, plus Gaussian noise:

$$\tilde{A}_\ell = A\,\mathrm{N}(H^{\ell}) + Z_\ell,\quad Z_\ell \sim \mathcal{N}(0, \sigma^2 I), \qquad \mathrm{N}(h) = h/\max(1, \|h\|_2)$$

Lemma G2. Fix everything released before layer ℓ. Under edge-DP, H^ℓ is then a function of public features, public labels and earlier noisy releases, so it is identical on neighbouring graphs. Adding or removing edge (u, v) changes rows u and v of $A \cdot N(H^\ell)$ by at most 1 each, so the ℓ_2 sensitivity is $\sqrt{2}$.

Each layer is a Gaussian mechanism with Rényi-DP α/σ^2, and one forward pass costs $L\alpha/\sigma^2$. That single pass yields predictions for every node, so per-node queries are free.

This is GAP's aggregation perturbation (Sajadmanesh et al., 2023) moved inside an in-context learner. GAP then trains a classifier per graph on cached noisy aggregates. DP-NodePFN replaces that training with a forward pass whose weights saw the same noise in the prior.

G2 requires four changes to NodePFN:

1. Drop symmetric normalization, which depends on private degrees. DPDGC shows standard graph convolutions need noise growing at least linearly with maximum degree.
2. Make the 32-member ensemble reuse one set of noisy aggregates, or pay for each member.
3. Noise or remove the edge-based feature-smoothing preprocessing.
4. Retrain on the prior with the same noise injected (noise-aware, Section 5). Without it, the noise is out of distribution.

G3. Edge-LDP: randomized response on adjacency. If every node pair is flipped with probability ρ = 1/(1 + e^ε), a cSBM stays a cSBM. Only its parameters move:

$$p' = \rho + (1 - 2\rho)\,p \quad (p \in \{p_{\mathrm{in}}, p_{\mathrm{out}}\}), \qquad p'_{\mathrm{in}} - p'_{\mathrm{out}} = (1 - 2\rho)(p_{\mathrm{in}} - p_{\mathrm{out}})$$

The noise-aware prior is therefore the same prior family with transformed parameters, which makes it easy to simulate. The cost is density. With N = 1,024 and ε = 2 (ρ ≈ 0.119), RR adds about 61,000 fake edges, against NodePFN's synthetic-graph average of about 12,700 real ones.

Proposition G3 (heuristic detectability threshold). Write a = Np_in and b = Np_out for K balanced communities. Apply the Kesten–Stigum condition $(a' - b')^2 > K(a' + (K - 1)b')$ to the randomized graph. Structure-only community signal then requires roughly:

$$N \;\lesssim\; \frac{(1-2\rho)^2\,(a-b)^2}{K^2\,\rho}$$

Example: K = 2, a = 20, b = 4 and ε = 2 give N below about 310. Larger graphs must take their signal from features, labels, or degree side-information, as in Blink (CCS 2023). Blink derives a Bayesian edge denoiser by hand. A noise-aware NodePFN would learn the full posterior predictive instead. Making this threshold rigorous is plan item T7.

G4. Node-DP: the hard case. Changing one node alters its features, its label and up to d of its edges. Every neighbour's aggregate moves, so sensitivity grows with a degree bound d_max (GAP's node-level variant; DPDGC). A DP-NodePFN route combines three parts:

- degree-bounded noisy aggregation, with the same truncation used in the prior;
- context sharding over disjoint node groups for raw features and labels;
- attention restricted to the teacher's own shard plus the shared noisy aggregates.

The total cost is the aggregation cost plus the vote cost, by composition. As in GAP, the analysis must state whether a node's own features may be used in its own prediction.

Privacy unit	DP-PFN mechanism	Change to NodePFN	What costs privacy
Label	graph-shard voting (G1) or RR + noise-aware PFN	none for voting	per answered query, or one student release

Privacy unit	DP-PFN mechanism	Change to NodePFN	What costs privacy
Edge (central)	noisy sum aggregation (G2)	sum aggregation, one ensemble pass, retrain on noisy prior	L Gaussian releases per forward pass, all nodes included
Edge (local)	RR on adjacency + noise-aware prior (G3)	retrain on transformed cSBM	the RR release only
Node	degree-bounded aggregation + shard voting (G4)	degree truncation, shard-local attention	aggregation plus votes, composed

7. Q2: can TabPFN and NodePFN serve as DP-PFN backbones?

Yes, both can, but they need different amounts of work.

- **TabPFN** works as-is inside the black-box mechanisms (C1–C4), once preprocessing is confined to a shard or made data-independent.
- **NodePFN** works as-is for label-DP. Edge-DP and node-DP need architectural changes and retraining on a noisy prior.

Requirement	TabPFN v2 / 2.5	NodePFN (ICLR 2026)
Weights from synthetic data only (Prop. 0)	yes for default checkpoints; avoid Real-TabPFN-2.5, which is fine-tuned on real datasets	yes, about 250,000 synthetic graphs
Context size	v2: 10,000 rows, 500 features; 2.5: 50,000 rows, 2,000 features	pretraining graphs have 1,024 nodes; attention is quadratic in nodes
How records interact	context rows attend to each other; queries attend only to context, not to each other	context self-attention plus a GCN branch mixing all nodes, test nodes included
Data-dependent preprocessing	quantile normalization, SVD features and other transforms fitted on the context	truncated SVD fitted per dataset; optional edge-based smoothing; normalized adjacency

Requirement	TabPFN v2 / 2.5	NodePFN (ICLR 2026)
Usable in C1–C4 without retraining	yes: each teacher fits preprocessing on its own shard	label-DP: yes (G1); edge- or node-DP: no
Needs retraining for	floored or clean-target targets (A5, A6); noise-aware use (Section 5)	edge-DP (G2), edge-LDP (G3), node-DP (G4), noise-aware use

Sources: TabPFN-2.5 report, TabPFN-2.5 overview, NodePFN, PFN attention masking.

TabPFN: what to watch.

1. **Preprocessing leaks.** In C1, each teacher fits its transforms on its own shard, so the shard's accounting covers them. Single-context mechanisms (A5, A6) and student-query generation (C4) need public feature bounds or a DP estimate of them.
2. **Ensembling is free.** Random feature and class permutations, preprocessing variants and the 64 learned "thinking rows" in 2.5 are all data-independent, as long as each member sees only its own shard.
3. **Shards fit the model's sweet spot.** Theorem C2 says the best shard size m sits where the learner's error curve flattens. TabPFN is strongest on small tables, which is exactly the regime C2 rewards.
4. **Queries do not see each other.** Many queries can share one pass with no cross-query mixing inside the model. Each answer is still a separate release in mode R1.
5. **Retraining is feasible at small scale.** Retraining TabPFN-2.5 from scratch is out of reach for most labs. Small TabPFN-style learners can be trained with open tooling such as TFM-Playground (used in a dataset-inference study). Continued pretraining of TabPFN-2.5 on synthetic-only tasks has also been demonstrated (TREDBench), and it keeps Proposition 0 intact.

NodePFN: what to watch.

1. **Label-DP works today.** Graph-shard voting (G1) needs no retraining, and every teacher keeps the full graph.
2. **Edge-DP needs the four G2 changes:** sum aggregation, shared noisy aggregates across the 32 ensemble members, no unprivatized smoothing, and noise-aware retraining.

3. **Scale is unclear.** Pretraining uses 1,024-node graphs. The paper reports results on graphs up to 48,921 nodes (Questions) without explaining how inference scales. Any subgraph sampling must be data-independent or counted, as with degree-bounded sampling under node-DP.
4. **No privacy or label-noise analysis.** The paper discusses neither. Its prior already spans homophily 0.1–0.9 plus Erdős–Rényi graphs, so the RR-shifted homophily in G3 stays inside a familiar prior family.

A cheap graph baseline from the tabular side. Treating nodes as rows with structural features is an established way to run TabPFN on graphs. Under edge-DP, those structural features must be private. Using G2′s noisy aggregates as features for an unmodified TabPFN gives a simple baseline: "GAP features + TabPFN". Comparing it with noise-aware DP-NodePFN isolates the value of noise-aware training.

8. Q3: advantages over prior DP tabular and graph methods

DP-PFN′s advantages come from three facts:

- the weights are free;
- there is nothing to train or tune on private data;
- the learner is near Bayes-optimal on small data, which is where DP hurts most.

Its weaknesses are the certification gap, per-query cost in mode R1, context-size limits, and node-DP. Each advantage below says whether it is proved here, holds by construction, or is a hypothesis for an experiment (Section 10).

Compared methods. On tables: DP-SGD on neural nets; DP boosting (DP-EBM); PATE and private prediction; DP synthetic data then non-private training (AIM, MST, PrivBayes, Tab-PE); DP in-context learning with LLMs (DP-ICL, DP-TabICL); and TabPATE. On graphs: GAP, ProGAP, DPDGC, node-level DP-SGD (Daigavane et al.), PATE-based PrivGNN, and local-DP methods (Blink, LPGNN, Solitude).

Advantage	Over which methods	Why	Status
No privacy spent on training	DP-SGD, DP boosting, GAP and ProGAP (node-DP training)	weights never see private data	proved (Prop. 0)

Advantage	Over which methods	Why	Status
No privacy spent on tuning	every trained DP model; tuning costs budget (Papernot & Steinke, 2022) and is often left unaccounted	nothing to tune per dataset; T, σ and η follow from ε by formula	by construction; E2 charges baselines for tuning
Public knowledge without public data	DP fine-tuning of pretrained models, where public pretraining data may overlap private data	the prior is synthetic and cannot contain private records	by construction
Smaller DP sample size on small data	PATE with trained teachers; DP-SGD	higher teacher accuracy at small shard size m lowers n*	proved (Thm. C2); measured in E3
Teachers cost one forward pass	PATE, PrivGNN	no per-teacher training	by construction; wall-clock in E2
Calibrated predictions from noisy data	DP-SGD models; naive use of DP synthetic data	the noise-aware target is the private-data PPD	proved under the prior (Thm. D1); tested in E4
Bayes-correct use of any simulatable DP release	pipelines that treat synthetic rows as real	the mechanism sits inside the prior	proved (Thm. D1, Prop. D2); tested in E4
Explicit, controllable prior alignment	DP-ICL with LLMs, which works mainly when tasks match pretraining (NeurIPS 2024 workshop)	the prior is designed, not inherited	by construction; misspecification ablation in E8
Graph label-DP keeps the whole graph	node-partitioned PATE on graphs	only labels are sharded (G1)	by construction; tested in E5
No per-graph training under edge-DP	GAP, ProGAP, DPDGC	one forward pass over noisy aggregates (G2)	by construction; tested in E6

Advantage	Over which methods	Why	Status
Learned denoising for edge-LDP	Blink's hand-derived Bayesian estimator	the PFN learns the posterior predictive for any mechanism it can simulate	hypothesis; tested in E6
Robustness across homophily under noise	DP-GNNs built around neighbour averaging	the prior spans homophily 0.1–0.9; the PFN can learn when noisy aggregates mislead	hypothesis; tested in E6

Where DP-PFN is weaker.

1. **Certification.** Only the black-box wrappers (Section 4) and noise-aware releases (Section 5) carry guarantees. The elegant one-draw mechanisms (Section 2) are exact only for the true PPD.
2. **Per-query cost.** A DP-SGD model answers unlimited queries once trained. DP-PFN in mode R1 pays per answer, unless it switches to R2 (student release) or R3 (noise-aware).
3. **Inference compute.** Vote and LogPool run T forward passes per query. Distilling into a student context (C4) removes this at deployment.
4. **Context limits.** Single-context mechanisms see at most 50,000 rows with TabPFN-2.5. Sharding handles larger n, but only for the wrapper mechanisms.
5. **Row-DP with continuous features.** Section 2 mechanisms need floored feature densities (Lemma A4), which may cost utility.
6. **Node-DP.** DP-PFN does not remove the degree-dependent sensitivity of graph aggregation (G4).
7. **Prior misspecification.** When real data sit far from the prior, every advantage shrinks. Adapting the prior to the private data would itself cost privacy.

9. Q4a: research plan, theory track

The direction has enough depth for three papers: tabular DP-PFN, graph DP-PFN, and a theory paper on exact and certified mechanisms. The theory track has nine questions. T4, T6 and the Section 3–5 results carry the first paper. T7 carries the graph paper. T1–T3, T5, T8 and T9 form the theory paper.

ID	Question	Target result	Builds on	Difficulty	Paper
T1	How good is a single posterior draw under a floored, misspecified likelihood?	utility rate for the clean-target release (A5, A6) at fixed ε, via posterior consistency under misspecification	Thm. A2, A5	medium	theory
T2	Central one-draw versus local RR plus a noise-aware PFN at equal ε	Bayes-risk comparison and the regimes where each wins	Sec. 2, Thm. D1	medium	theory
T3	Can a constrained PFN be certified with s(θ, n) = O(C/n)?	row-sensitivity bound for bounded-logit or L2 attention of depth L; a propose-test-release variant	Prop. B3	hard	theory
T4	When does LogPool beat Vote?	accuracy bounds as a function of teacher calibration; condition on T and the expected log-likelihood ratio	Mech. C1, C3	medium	tabular
T5	Which release mechanism should one use for a given prior?	choose M to minimize I(y*; D | x*, S) at fixed ε; a Fano-type lower bound valid for exchangeable priors	Thm. D1	hard	theory
T6	Explicit DP sample complexity for DP-PFN	n*(ε) from Thm. C2 with learning curves e(m) for linear, GP and SCM priors; separation from DP-SGD rates when n is small	Thm. C2	medium	tabular
T7	When does edge-LDP destroy graph structure?	a rigorous version of Prop. G3 for the contextual SBM, using the Deshpande et al. (2018) threshold with features	Prop. G3	medium	graph
T8	Node-DP accounting for DP-NodePFN	composition of degree-bounded aggregation and shard voting with shard-local attention; an accuracy lower bound	G4	hard	theory or graph
T9	Can a PFN act as an amortized Bayes-optimal auditor?	a PFN trained on (dataset, membership bit, output) gives valid one-prior lower bounds on ε, consistent with DP's Bayesian semantics	Sec. 3	medium	theory

Order of work.

1. **First:** T4 and T6. They are short proofs and directly shape the tabular experiments (E2, E3).
2. **In parallel:** turn Propositions B2 and B3 into a clean "no certification from loss" theorem. It is the conceptual anchor of the first paper.
3. **Graph paper:** T7, because Proposition E3 already gives the shape of the result.
4. **Theory paper:** T1, T2, T3, T5, T8 and T9 once the empirical picture is clear.

10. Q4b: research plan, experiments

Eight experiments test the theory and the claimed advantages. E1 decides early whether the exact-PPD mechanisms are usable in practice. E2–E4 make up the tabular paper, E5–E6 the graph paper, and E7–E8 run across both.

Shared setup.

- **Models.** Small PFNs trained in-house on four kinds of prior:
  - closed-form priors;
  - floored priors with η as an input;
  - clean-target variants (A5, A6);
  - noise-aware variants (label RR, Gaussian marginals, noisy graph aggregates).

  Released synthetic-only TabPFN checkpoints serve as teachers. NodePFN serves as is (label-DP) and retrained with sum aggregation and noise (edge-DP).
- **Tabular data.** OpenML-CC18 and the TabArena-lite classification subset, plus privacy-relevant tables: Adult, ACS Income and Employment (folktables), credit default, bank marketing, and Diabetes 130-US hospitals. Subsample to $n \in \{500, 2k, 10k, 50k\}$.
- **Graph data.** Homophilous: Cora, Citeseer, Pubmed, Amazon-Photo, Coauthor-CS. Heterophilous: Chameleon, Squirrel, Actor, Amazon-Ratings, Tolokers. DP-GNN standards: Facebook, Amazon, LastFM. Reddit only if inference scales.
- **Privacy levels.** $\varepsilon \in \{0.5, 1, 2, 4, 8\}$, with δ below $1/n$. Report label-, row-, edge- and node-level results separately.
- **Metrics.** Accuracy and AUROC; NLL and expected calibration error; total ε including tuning; wall-clock and forward passes; empirical privacy from attacks (E7).

Experiments.

1. **E1: exact versus amortized privacy (go/no-go for Section 2).** Train PFNs on floored Beta–Bernoulli and Dirichlet–categorical priors, where the exact privacy loss is known.

- Measure $\Lambda(D)$ on prior-typical datasets versus adversarially searched ones (outliers, out-of-support rows).
- Search for the worst neighbouring pair to lower-bound $s(\theta, n)$.
- Tests Prop. B2 and Observation B5. If the gap is small and stable, Section 2 mechanisms earn an audited empirical role; otherwise they stay test beds.

2. **E2: tabular private prediction (main table).** Compare DP-PFN-Vote, LogPool and Student with: non-private TabPFN and XGBoost (upper bounds); DP-SGD MLP; DP logistic regression; DP-EBM; PATE with trained teachers; TabPATE; and DP synthetic data (AIM, MST, Tab-PE) with TabPFN or XGBoost downstream.
   - Run every trained baseline twice: with tuning charged to the budget, and tuning-free.
3. **E3: sample-complexity curves.** Sweep n and shard size m at fixed ε. Measure $e(m)$, predict the best m and n^* from Theorem C2, and compare with what is observed. Repeat with trained teachers to show the PFN's small-m advantage directly.
4. **E4: noise-aware DP-PFN.** For label RR, noisy marginals and DP synthetic data, compare naive TabPFN on the released data with the noise-aware PFN.
   - Report NLL and ECE alongside accuracy. Prop. D2 predicts under-confidence for RR and over-confidence for synthetic rows.
   - On small problems, compare with the per-problem neural posterior estimators of Xiong, Ju & Zhang.
5. **E5: graph label-DP.** Compare graph-shard voting with NodePFN (mechanism G1) against: label RR + GCN; a node-partitioned PATE GNN; and RR + noise-aware NodePFN. Report homophilous and heterophilous graphs separately.
6. **E6: graph edge-DP and edge-LDP.**
   - **Central edge-DP:** DP-NodePFN (mechanism G2) versus GAP, ProGAP, DPDGC and "GAP features + TabPFN".
   - **Edge-LDP:** noise-aware NodePFN versus Blink and RR + GCN. Plot accuracy against N to test the threshold of Prop. G3.
7. **E7: audits.** For every mechanism, run membership attacks and report the empirical ε they imply.
   - **Tabular:** LiRA-style shadow contexts, attention-based AMIA, and one-run auditing.
   - **Graphs:** link-inference attacks.

- **Why this is cheap:** shadow "models" for a PFN are forward passes with and without the target, so strong audits cost little. This is itself a methodological contribution.

8. **E8: ablations and stress tests.**
   - floor η, number of teachers T, and Vote versus LogPool;
   - number of answered queries;
   - student-context size N (C4) versus the one-draw release (A6);
   - ensemble reuse in DP-NodePFN;
   - prior misspecification: label noise, heavy-tailed features, and homophily outside the prior's range.

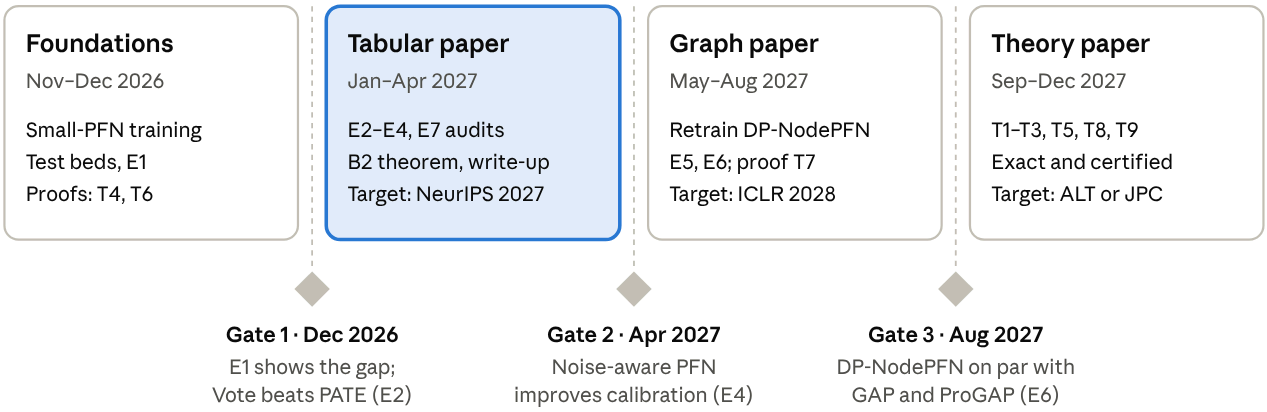
Go/no-go checks. Each gate in Section 11 is tied to an experiment:

- Gate 1 needs E1 to show the predicted gap, and DP-PFN-Vote to beat trained-teacher PATE on small tables.
- Gate 2 needs the noise-aware PFN to improve calibration over naive use (E4).
- Gate 3 needs DP-NodePFN to be competitive with GAP and ProGAP under edge-DP on at least the small graphs (E6).

11. Timeline, risks and venues

The plan runs about 14 months from November 2026. The tabular paper comes first because it needs no retraining of large models and already has a competitor in TabPATE.

Three papers in four phases, each ending at a go/no-go gate



roadmap · 4 phases, 3 gates

Each gate decides whether the next phase proceeds as planned or is cut back. Phases will overlap in practice. Conference deadlines shown are typical and need confirming.

Risk	Likelihood	Mitigation
The amortization gap makes the one-draw mechanisms unusable	high	the design already rests on black-box wrappers and noise-aware releases; Section 2 stays as theory and as the audit target
TabPFN-scale retraining costs too much	medium	small PFNs for the new targets; synthetic-only continued pretraining of released checkpoints
Real data sit far from the prior	medium	measure it in E8; spend a small DP budget on model selection where needed
NodePFN does not scale, or its code is unavailable	medium	start with small and medium graphs; reimplement the GCN-plus-attention design; use data-independent subgraph batching
Concurrent work; TabPATE already covers PATE-style tabular ICL	high	lead with what TabPATE lacks: Theorems B2, C2 and D1, LogPool, noise-aware PFNs and graphs; submit the tabular paper first
Baselines judged unfairly	medium	report results with tuning both charged and uncharged; release the accounting code

Venues.

- **Machine learning:** NeurIPS, ICML, ICLR.
- **Security and privacy:** USENIX Security, CCS, IEEE S&P, PETS, SaTML.
- **Theory:** COLT, ALT, Journal of Privacy and Confidentiality.
- **Graph learning:** LoG, KDD, WWW.

References

Sources consulted for this document.

- Müller et al., Transformers Can Do Bayesian Inference (PFNs), ICLR 2022
- Choi, Kim, Kang & Park, Learning Posterior Predictive Distributions for Node Classification from Synthetic Graph Priors (NodePFN), ICLR 2026
- Prior Labs, TabPFN-2.5 report and overview
- GraphPFN, used here for PFN attention masking
- Wahdany, Jagielski, Cresswell, Dziedzic & Boenisch, TabPATE, ICML 2026 workshop
- Carvalho & Cordy, Privacy Vulnerabilities of Attention Layers in Tabular Foundation Models (AMIA), 2026
- Risk In Context: privacy leakage of foundation models in tabular synthesis, 2025
- Xiong, Ju & Zhang, Simulation-based Bayesian Inference from Privacy Protected Data, TMLR 2025
- Panda et al., Differentially Private In-Context Learning, 2023
- A Cautionary Tale on the Evaluation of DP In-Context Learning, NeurIPS 2024 workshop
- Carey et al., DP-TabICL, 2024
- Tran et al., DP Synthetic Data via APIs 4: Tabular Data (Tab-PE), ICML 2026
- Sajadmanesh et al., GAP: DP GNNs with Aggregation Perturbation
- Sajadmanesh & Gatica-Perez, ProGAP, WSDM 2024
- Chien et al., DPDGC: DP Decoupled Graph Convolutions, NeurIPS 2023
- Zhu, Tan & Xiao, Blink: Link Local DP in GNNs via Bayesian Estimation, CCS 2023
- Kim, Papamakarios & Mnih, The Lipschitz Constant of Self-Attention
- Dataset inference artefacts for tabular foundation models (TFM-Playground)
- TREDBench: synthetic-only continued pretraining of TabPFN-2.5

Background references (cited from memory, not re-checked here).

- **Posterior sampling as DP:** Dimitrakakis et al. (2014), Robust and private Bayesian inference; Wang, Fienberg & Smola (2015), Privacy for free; Geumlek, Song & Chaudhuri (2017), Rényi DP mechanisms for posterior sampling.
- **Inference from privatized data:** Williams & McSherry (2010); Bernstein & Sheldon (2018); Ju et al. (2022), Data augmentation MCMC from privatized data.
- **Private prediction and teacher ensembles:** Dwork & Feldman (2018), Privacy-preserving prediction; Papernot et al. (2017, 2018), PATE and GNMax.

- **DP tools:** Balle, Barthe & Gaboardi (2018), amplification by subsampling; Papernot & Steinke (2022), hyperparameter tuning with Rényi DP; Kasiviswanathan & Smith (2014), Bayesian semantics of DP; Steinke, Nasr & Jagielski (2023), one-run privacy auditing.
- **Bayesian aggregation and predictive resampling:** Tresp (2000), Bayesian committee machine; Fong, Holmes & Walker (2023), martingale posteriors.
- **Graphs:** Deshpande et al. (2018), contextual SBM thresholds; Daigavane et al. (2021), node-level DP GNNs; Olatunji et al., PrivGNN; Sajadmanesh & Gatica-Perez (2021), LPGNN.
- **DP tabular baselines:** McKenna et al., AIM and MST; Zhang et al., PrivBayes; Nori et al. (2021), DP-EBM.